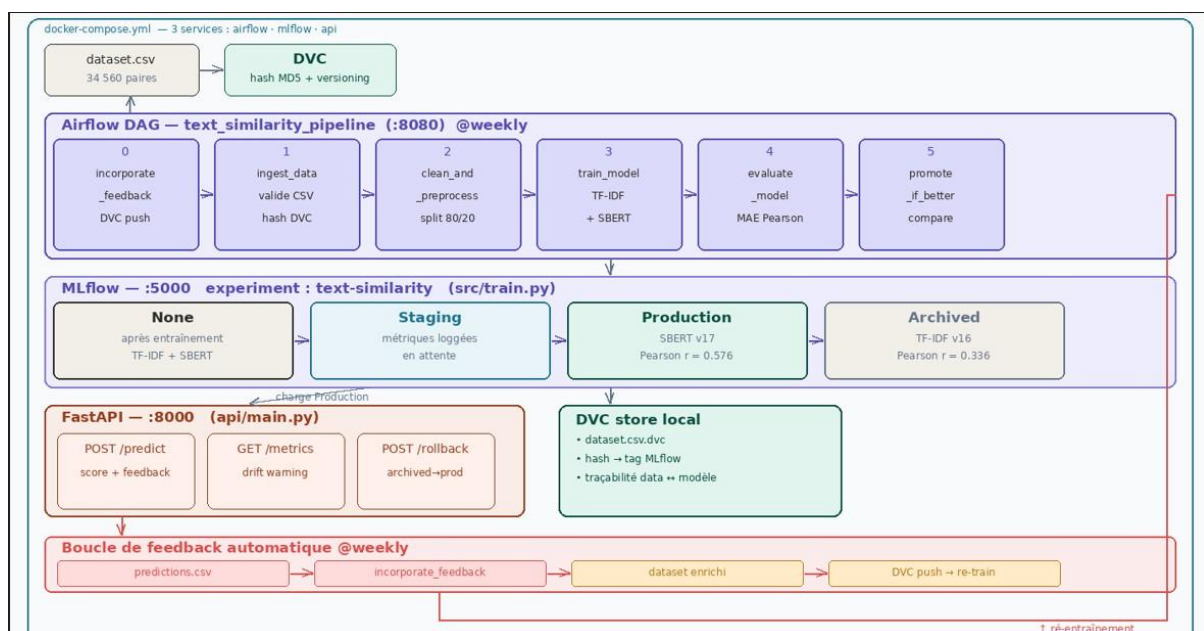


Architecture MLOps

MLOPS_LIPSTIP — Explication étape par étape

Vue d'ensemble

Ce document décrit l'architecture du pipeline MLOps développé pour le test technique Lipstip. Le système apprend automatiquement à mesurer la ressemblance sémantique entre deux textes. Il tourne entièrement en local via Docker Compose, sans aucune dépendance à un service externe. Chaque étape est orchestrée par Airflow, tracée par MLflow et versionnée par DVC.



Étape 0 — La donnée brute — dataset.csv

Tout commence par un fichier CSV de 34 560 paires de phrases séparées par « ; » avec trois colonnes : `sentence_1`, `sentence_2`, `similarity_score` [0,1]. C'est la matière première de tout le pipeline. Le fichier est placé manuellement dans le dossier `data/` avant le premier lancement.

└─ **dataset.csv** → **sentence_1 ; sentence_2 ; similarity_score**

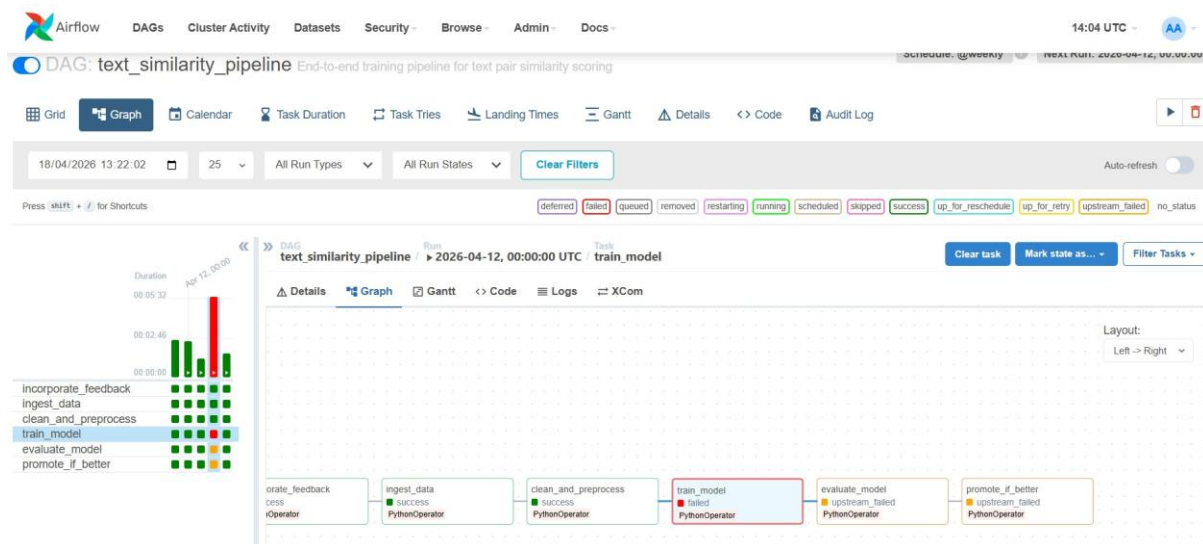
Étape 1 — DVC — versioning des données

Avant toute chose, DVC calcule une empreinte unique (hash MD5) du dataset. Si une seule ligne du fichier change, cette empreinte change. Elle est enregistrée dans un fichier .dvc et sera attachée à chaque run MLflow. Résultat : on sait toujours quelle version exacte des données a produit quel modèle. C'est la traçabilité données - modèle.

| **dataset.csv → DVC hash MD5 → tag MLflow "dataset_hash=abc123"**

Étape 2 — Airflow démarre le pipeline automatiquement

Chaque semaine à l'heure programmée (@weekly), Airflow réveille le DAG text_similarity_pipeline. C'est le chef d'orchestre, il décide dans quel ordre les 6 tâches s'exécutent et arrête tout si une tâche échoue. Les tâches s'enchaînent de gauche à droite avec des dépendances séquentielles strictes.



Étape 3 — Tâche 0 : incorporate_feedback

Première tâche du DAG. Elle vérifie si l'API a enregistré de nouvelles prédictions depuis la dernière exécution dans predictions.csv. Si oui, elle les ajoute au dataset principal. DVC versionne ce nouveau dataset enrichi. Le modèle suivant s'entraînera donc sur des données plus représentatives de l'usage réel. C'est la boucle d'amélioration continue.

| **predictions.csv → dataset.csv enrichi → DVC push (nouvelle version)**

Étape 4 — Tâche 1 : ingest_data

Le CSV est lu et validé rigoureusement. Trois vérifications sont effectuées :

- Les trois colonnes (sentence_1, sentence_2, similarity_score) existent-elles ?
- Les scores sont-ils tous compris entre 0 et 1 ?
- Y a-t-il des valeurs nulles ou des textes vides ?

Si une vérification échoue, Airflow marque la tâche en rouge et toute la chaîne s'arrête immédiatement. Les tâches suivantes passent en « upstream_failed ». C'est le gardien de la qualité des données.

| **dataset.csv → validation schéma + scores → OK ou FAIL (stop pipeline)**

Étape 5 — Tâche 2 : clean_and_preprocess

Les données valides sont nettoyées en plusieurs opérations :

- Suppression des doublons sur (sentence_1, sentence_2)
- Suppression des lignes nulles
- Normalisation du texte : strip() + lowercase()

Puis le dataset est découpé en 80 % pour entraîner les modèles et 20 % pour les évaluer. Ce découpage est fixe grâce à random_state=42, il est identique à chaque run, ce qui rend les comparaisons de modèles équitables et reproductibles.

| **dataset.csv → nettoyé → train.csv (80%) + test.csv (20%)**

Étape 6 — Tâche 3 : train_model

Les deux modèles s'entraînent en parallèle sur les 80 % de données d'entraînement.

- TF-IDF construit un vocabulaire et compare les phrases par leurs mots en commun via la similarité cosinus. Simple, rapide (37 secondes), interprétable.
- SBERT (all-MiniLM-L6-v2) encode chaque phrase en un vecteur de 384 dimensions qui capture le sens profond du texte. Deux phrases synonymes auront des vecteurs proches même sans mot en commun. Plus lent (54 secondes) mais sémantiquement puissant.

Chaque modèle ouvre un run MLflow avec ses paramètres loggués. Les deux versions passent au stage « None » dans le Registry.

| **train.csv → TF-IDF (MLflow None) + SBERT (MLflow None)**

Étape 7 — Tâche 4 : evaluate_model

Les deux modèles sont testés sur les 20 % de données mis de côté. Quatre métriques sont calculées et loggées dans MLflow :

- MAE (Mean Absolute Error) : erreur moyenne — intuitif, même unité que le score
- MSE (Mean Squared Error) : pénalise les grandes erreurs
- Pearson r : corrélation linéaire — critère principal de promotion
- Spearman r : corrélation de rang — robuste au déséquilibre du dataset (68 % des scores < 0.1)

Résultats réels obtenus : TF-IDF (MAE=0.207, Pearson=0.336) / SBERT (MAE=0.259, Pearson=0.576). Les deux modèles passent au stage Staging dans le Registry.

test.csv → TF-IDF : Pearson=0.336 | SBERT : Pearson=0.576 → Staging

Étape 8 — Tâche 5 : promote_if_better

C'est la tâche de décision finale. Elle compare le Pearson r des deux modèles entre eux, puis compare le gagnant avec le modèle déjà en Production. Deux cas possibles :

- Si le Pearson r est meilleur → le modèle est promu en Production, l'ancien passe en Archived.
- Si non → le nouveau part directement en Archived. L'ancien reste en Production.

Dans ce projet : SBERT (0.576) bat TF-IDF (0.336) → SBERT devient la version 17 en Production, TF-IDF devient la version 16 en Archived. Ce statut est visible dans l'interface MLflow sous l'onglet Models.

SBERT Pearson=0.576 > TF-IDF 0.336 → SBERT = Production v17 | TF-IDF = Archived v16

mlflow2.10.0

Experiments

Models

text-similarity >

sbert

Run ID: 43386c9188e8462685164d411c1be154

Date: 2026-04-18 14:22:37

Source: airflow

User: airflow

Duration: 54.5s

Status: FINISHED

Lifecycle Stage: active

> Description [Edit](#)

> Parameters (5)

> Metrics (4)

Name	Value
mae	0.2591424045469113
mse	0.09796842235371303
pearson_r	0.5765220840167276
spearman_r	0.5364465621038841

text-similarity > **tfidf**

Run ID: 88f588ef89ac482fbb28298b949110c8 Date: 2026-04-18 14:22:19 Source: airflow User: airflow

Duration: 37.3s Status: FINISHED Lifecycle Stage: active

> Description [Edit](#)

> Parameters (6)

▼ Metrics (4)

Name	Value
mae 🔗	0.2072096548779898
mse 🔗	0.1603385360118501
pearson_r 🔗	0.336357446070499
spearman_r 🔗	0.20043439101023236

mlflow 2.10.0 Experiments Models

Registered Models > text-similarity >

Version 16

Registered At: 2026-04-18 14:22:24 Last Modified: 2026-04-18 14:23:38 Source Run: [tfidf](#)

Stage (deprecated): Archived ⓘ

> Description [Edit](#)

> Tags

▼ Schema

Name	Type
------	------

mlflow 2.10.0 Experiments Models

Registered Models > text-similarity >

Version 17

Registered At: 2026-04-18 14:22:38 Last Modified: 2026-04-18 14:23:38 Source Run: [sbert](#)

Stage (deprecated): Production ⓘ

> Description [Edit](#)

> Tags

▼ Schema

Name	Type
------	------

No schema. See [MLflow docs](#) for how to include input and output schema with your model.

Étape 9 — FastAPI — serving du modèle en production

Dès que le pipeline s'est exécuté, l'API FastAPI charge automatiquement le modèle marqué Production depuis MLflow. Il est gardé en mémoire — pas rechargé à chaque requête. Trois endpoints sont disponibles :

- POST /predict : reçoit deux phrases, encode avec SBERT, calcule la similarité cosinus, retourne le score. Chaque prédiction est sauvegardée dans predictions.csv.
- GET /metrics : surveille les scores moyens et signale un drift si les prédictions dérivent anormalement (alerte si score moyen hors de la plage [0.05, 0.80]).
- POST /rollback : recharge instantanément la version Archived en Production sans redémarrer le service. Utile si le modèle promu se comporte mal.

requête {s1, s2} → SBERT encode → cosine similarity → score → predictions.csv

Étape 10 — Boucle de feedback — le cycle s'auto-améliore

Chaque prédiction est sauvegardée dans predictions.csv. La semaine suivante, Airflow redémarre le DAG automatiquement. La tâche 0 (incorporate_feedback) lit les prédictions accumulées, les ajoute au dataset principal, DVC crée une nouvelle version du dataset enrichi, et tout le cycle recommence. Le modèle suivant s'entraîne sur des données qui incluent l'usage réel de l'API — il s'améliore progressivement sans aucune intervention manuelle.

predictions.csv → dataset enrichi → DVC v2 → nouveau run → modèle amélioré